



Rapport d'analyse : projet VeloScope

Équipe 14

Étudiants :

BONAVERO Méline
BOYCE Eddy
CHATONNIER Eliot
LEJARS Samuel
MOINERAUD-DOUCET Alix

Encadrant :

RIVIERE André

Table des matières

1	Cahier des charges	1
1.1	Utilisateurs	1
1.2	Fonctionnalités de base	1
1.3	Déroulé de la navigation	2
2	Modélisation	4
2.1	Architecture de l'API	4
2.1.a	Frontend	5
2.1.b	Backend	6
2.1.c	Couche persistente	6
2.1.d	Outils de développement	6
2.2	Modèle de données	7
2.3	Classes	7
2.4	Description des endpoints	8
2.5	Diagramme de séquence	8
3	Calendrier prévisionnel	8
	Annexes	9
	Bibliographie	10

1 Cahier des charges

L'objectif du projet est produire une application donnant des informations sur la disponibilité des vélos dans une ville. Au-delà de la seule disponibilité en temps réel par station, l'application doit également fournir des indicateurs de qualité. Elle donnera notamment un indicateur de fiabilité, qui pourra potentiellement être visualisé grâce à une carte.

1.1 Utilisateurs

Deux types d'utilisateurs sont à distinguer. Il y aura tout d'abord les utilisateurs «réguliers» de l'application, qui devront créer un compte puis se connecter pour accéder à l'application. Une fois connecté, ils pourront accéder aux données de l'application et gérer leurs stations favorites. Ensuite, il y aura des comptes «administrateurs» qui pourront gérer les comptes utilisateurs.

1.2 Fonctionnalités de base

L'application possédera plusieurs fonctionnalités de base. Voici un diagramme de cas d'utilisation, que nous détaillons ci-dessous :

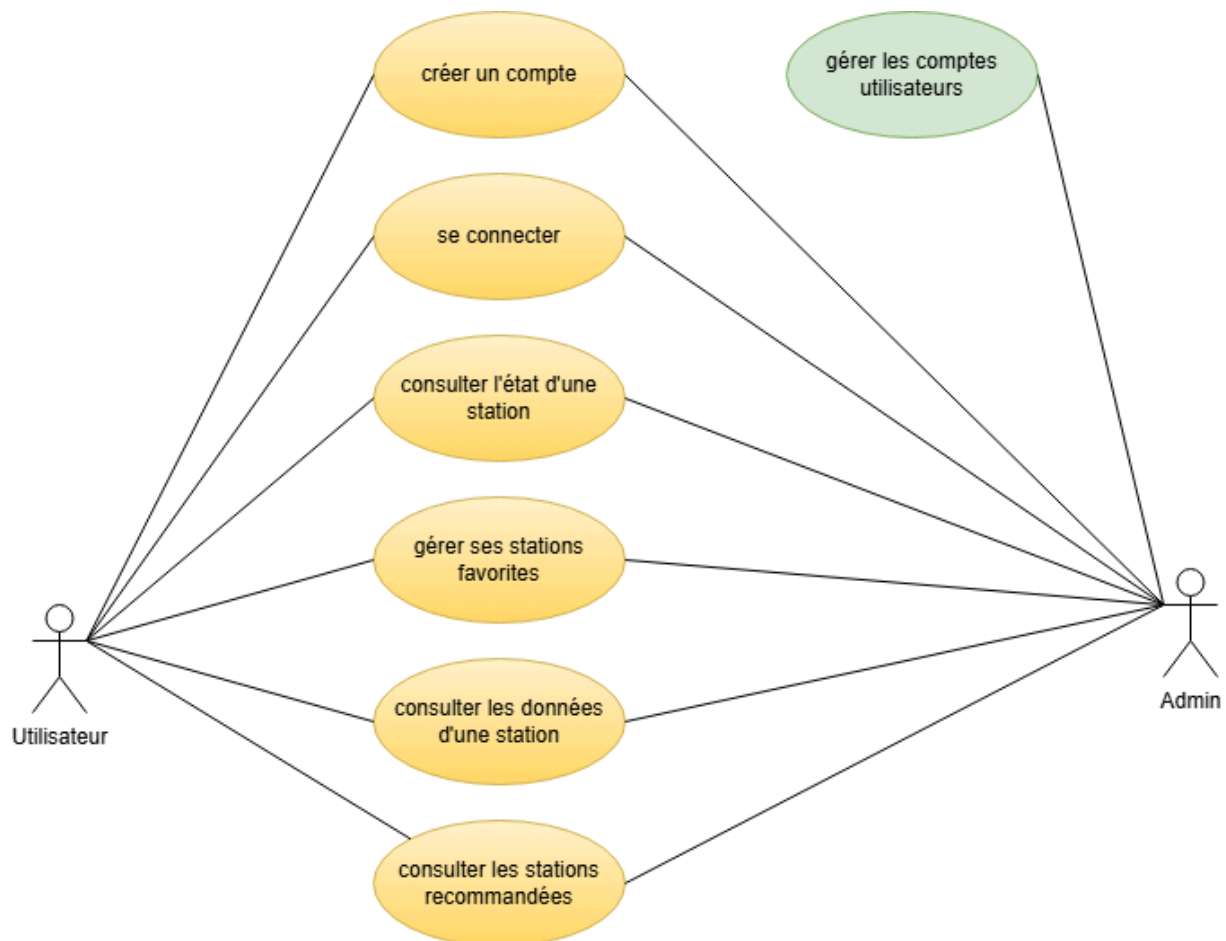


Figure 1: Diagramme de cas d'utilisation

Collecter des données via une API

La collecte des données concernant la disponibilité et l'état des stations de vélos se fera via une API externe. La collecte se fera à intervalle de temps régulier (toutes 15 min). Elle permettra de nourrir une

base de données, hébergée ..., qui contiendra notamment le nom des stations, leurs localisations, leurs états et la disponibilité en terme de vélos et d'emplacement.

Consulter des données

Les utilisateurs pourront consulter, pour la station de leur choix, diverses informations. Premièrement, les données fournis par l'API (état de fonctionnement, disponibilité). Deuxièmement, des données calculées grâce à l'accumulation des données collectés (taux de remplissage, évolution de la disponibilité, durée moyenne des épisodes de saturation et de pénurie).

Fournir un score de fiabilité

Pour chaque station, l'application calculera et fournira un score de fiabilité. La construction de ce score est détaillée ci-dessous :

Recommander une station

L'application permettra de recommander des stations à proximité de l'utilisateur. Cette recommandation se basera à la fois sur la distance à l'utilisateur, sur l'historique de station (notamment via le score de fiabilité) et sur son historique récent.

1.3 Déroulé de la navigation

Voici ci-dessous un diagramme d'activité modélisant les différentes étapes de navigation dans l'appliation :

Diagramme d'Activité

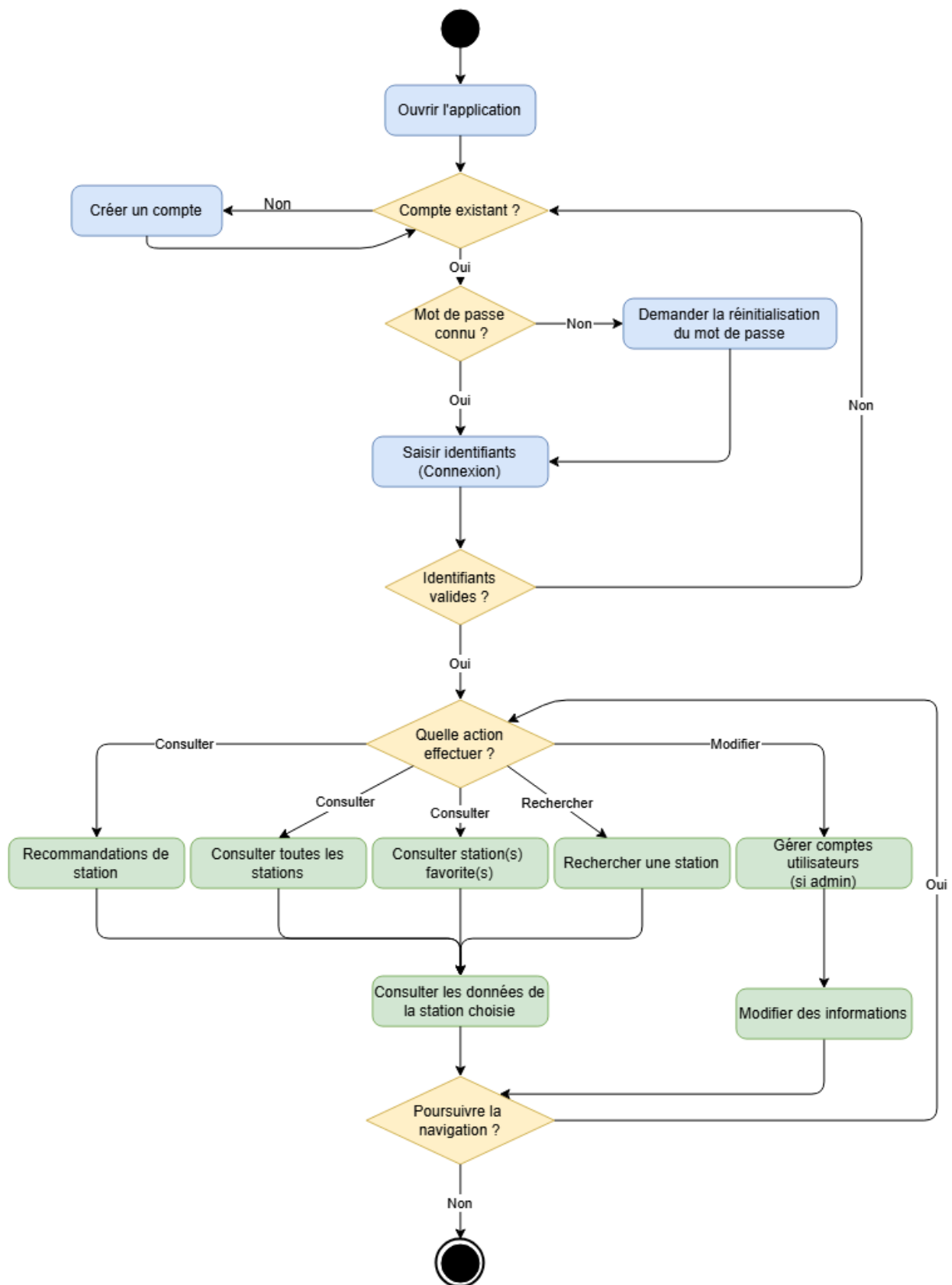


Figure 2: Diagramme d'activité

Gestion de compte

La première partie de ce diagramme correspond à la partie de gestion de compte de l'utilisateur. Elle lui permet de créer un compte si il en a pas, de modifier son mot de passe en cas d'oubli ou bien de se connecter à son compte. Seul l'administrateur dispose après connexion à son compte de la possibilité d'influer sur les comptes des utilisateurs "réguliers".

Fonctionnalité principales

L'ensemble des fonctionnalités de l'application sont dans la deuxième partie de ce diagramme. L'ensemble des options (recommandations, liste, favoris et recherches) mènent à la consultation des données liées à une seule station. L'utilisateur a ensuite le choix de quitter l'application ou de poursuivre la navigation en retournant à l'étape du choix d'action.

2 Modélisation

2.1 Architecture de l'API

L'API sera constituée de plusieurs couches afin de repsecter le principe de séparations des responsabilités. Voici un schéma d'ensemble de l'architecture logiciel envisagée, que nous détaillons dans la suite du dossier.

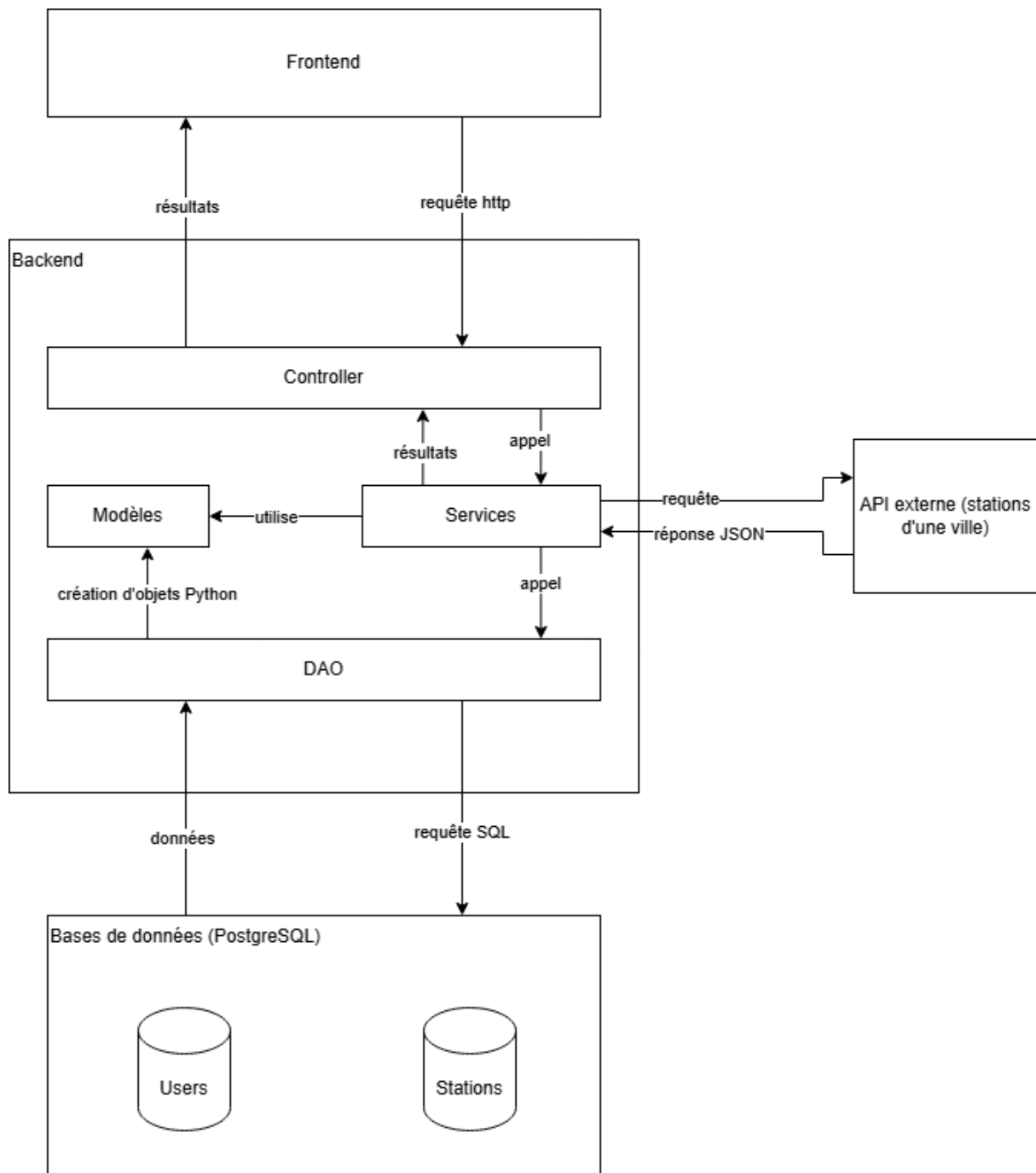


Figure 3: Diagramme d'architecture logiciel

2.1.a Frontend

Le *frontend* correspond à l'interface homme-machine. Il prendra la forme d'une application interactive accessible via une page Web.

L'interface se voudra simple et fonctionnelle : elle se présentera sous la forme d'une page HTML. Un menu à gauche permettra à l'utilisateur de sélectionner la fonctionnalité qu'il souhaite. Les différentes fonctionnalités disponibles porteront les noms suivants :

- rechercher une station,
- mes stations favorites,
- recommandation,

- gérer mon compte,
- gérer les comptes utilisateurs (n'apparaîtra que pour les administrateurs).

La gestion des stations favorites se fera de la manière suivante : l'utilisateur pourra ajouter de nouvelles stations lors de la consultation de la page d'une station. Il pourra ensuite consulter ses stations favorites ou les supprimer dans l'onglet "mes stations favorites".

2.1.b Backend

Le *frontend* enverra des requêtes http de la part du *backend*, qui sera chargé de traiter ces requêtes et de renvoyer la réponse attendue. Ces requêtes concerneront soit la gestion des utilisateurs (création de compte par exemple), soit la gestion des stations de vélos (consultation de la page d'une station par exemple). Le backend sera séparé en plusieurs couches.

Tout d'abord, une couche *controller* recevra les requêtes fournies par le frontend lors de l'utilisation de l'API par un usager. Cette couche aura pour but de convertir et transférer ces requêtes à la couche suivante, celles des services.

La couche *service* est le coeur fonctionnelle de l'API. C'est dans cette couche que seront regroupées les fonctions apportant une valeur ajoutée à notre API. Il s'agira par exemple des fonctions permettant de calculer les indicateurs souhaités et le score de fiabilité pour chaque station de vélos. Pour cela, la couche *service* a besoin de données. Ces services feront donc appel à une couche *DAO* (Data Access Object) afin de communiquer avec les bases de données.

La couche *DAO* recevra les demandes de données de la part de la couche *service*. Elle sera chargée de convertir ces demandes en requêtes SQL. Ces requêtes seront alors traitées par la couche persistente : celle des bases de données. La couche *DAO* aura alors pour mission de convertir en objets python (par exemple en objet *Utilisateur* ou *Station*, cf la section "Modèle de données" pour plus de détails). Ces objets seront ensuite utilisés par la couche *service* afin de calculer les indicateurs qui seront transmis à la couche *controller* afin de les afficher à l'utilisateur dans l'interface.

2.1.c Couche persistente

La couche persistente sera constituée des bases de données qu'utilisera notre API. Deux bases de données sont prévues : l'une pour les données utilisateurs, l'autre pour les données accumulées sur les stations de vélos.

2.1.d Outils de développement

Le projet sera réalisé en Python. Nous utiliserons plusieurs outils et packages fournis pour ce langage. Voici le détail de ces outils pour chaque couche :

- le framework [streamlit](#) pour le *frontend* (afin de construire l'application Web interactive),
- le framework [FastAPI](#) pour le *backend*,
- le package [psycopg2](#) pour la couche *DAO* (afin de convertir les instructions Python en requêtes lisibles par les tables PostgreSQL).

De plus, afin de garantir une maintenance aisée, nous tenterons de respecter les bonnes pratiques de codage tant que faire se peut. Dans ce but, nous serons amenés à utiliser les outils suivants :

- Ruff pour le formatage du code,
- Pytest afin d'effectuer les tests unitaires,
- Github pour héberger et versionner notre projet.

2.2 Modèle de données

Afin que le modèle de données soit cohérent et pérenne, nous veillerons à respecter les trois premières formes normales.

Concernant les utilisateurs, une seule table “Users” sera nécessaire. Elle contiendra les données des utilisateurs : identifiant, pseudonyme de l'utilisateur, mot de passe hashé, statut (utilisateur simple ou administrateur).

Pour les stations, les données à stocker sont les suivantes : identifiant et nom de la station, coordonnées géographiques, état de la station, nombre d'emplacements, nombre d'emplacements libres, nombre de vélos disponibles (potentiellement électriques), date et heure. Le stockage des données au cours du temps nécessite d'être attentif aux formes normales. En effet, un stockage naïf des données nécessiterait une clé composite (id_station, date, heure). Cela créerait plusieurs problèmes :

- l'attribut nom_station serait en dépendance fonctionnelle avec id_station et donc avec seulement une partie de la clé primaire (contradiction avec le principe de la deuxième forme normale) ;
- l'attribut coordonnees_geo serait en dépendance fonctionnelle avec nom_station (contradiction avec le principe de la troisième forme normale).

Il faudra donc deux tables. La première sera la relation **Stations**. Ses attributs stockeront les données concernant les stations elles-mêmes : l'identifiant de la station (clé primaire), son nom et ses coordonnées géographiques. La seconde sera la relation **StationRecords**, dont les attributs stockeront les données fournies par l'API externe. Voici les données stockées : l'identifiant de la station (attribut `id_station`), date et heure de l'enregistrement (attribut `date`), le nombre d'emplacements de la station (`nb_spaces`), le nombre d'emplacements libre de la station (`nb_available_spaces`), le nombre de vélos disponibles (`nb_available_bikes`) et le nombre de vélos électriques disponibles (`nb_available_ebikes`).

Enfin, l'un des attendus du projet est de permettre aux utilisateurs de pouvoir gérer leurs stations favorites. Cela se fera via une table d'association “Favorite” qui permettra de lier un utilisateur à ses stations favorites.

[Diagramme UML avec les données suivantes :]

2.3 Classes

D'après l'architecture logicielle, le projet nécessitera différents types de classes. Tout d'abord, il faudra implémenter les classes modèles “User”, “Station”, et “StationRecord”. Chacune de ces classes nécessitera une classe DAO propre afin de pouvoir faire la correspondance entre ces objets et les tables de données correspondantes. Il faut également implémenter des classes controller permettant de gérer les différents endpoints de l'API.

Chacune des trois classes métiers prévues aura une classe “service” associée. Pour les classes “Users” et “StationRecord”, les services permettront essentiellement de faire la jonction entre la couche “controller” et la couche “DAO” : ce seront donc des services purement techniques. La réelle plus-value de l'API se fera au niveau des services associés à la classe “Station”, notamment via le calcul d'indicateurs. Afin d'éviter d'avoir une classe avec de trop nombreuses méthodes, et dans un but de factorisation du code, les services associés à la classe “Station” seront séparés en deux classes. Tout d'abord une classe “StationService” qui se concentre sur la logique métier, et qui permettra d'obtenir les indicateurs via une méthode `get_stats` qui fera appel à la classe `StationCalculationService`. Cette dernière sera chargée de calculer les indicateurs statistiques souhaités (taux de remplissage, disponibilité au cours du temps, pourcentage du temps passé vide ou saturé, temps moyen des épisodes de saturation et de pénurie, score de fiabilité).

2.4 Description des endpoints

endpoints utilisateurs: consulter une station ajouter une station en favori obtenir la liste des stations
afficher les stations recommandées créer un compte se connecter ajouter une station en favori
supprimer une station des favoris

endpoints pour administrateur : changer un mdp changer un nom utilisateur supprimer utilisateur

Commande http GET(récupération)/PUT(mis à jour)/POST(insertion)/DELETE(suppression)/PATCH

2.5 Diagramme de séquence

3 Calendrier prévisionnel

diagramme de Gantt

Annexes

Bibliographie